DMS - RAG Project

Project Setup

Frontend

This project requires Node.js 22 and npm to run.

Recommendations:

- *use nvm to manage Node.js installs. Guide here*

---

1. Clone the project from the github repo.
2. In the project root directory, open a terminal and install the dependencies using

```
npm install
# or
bun install
```

3. For the first time running this application, run

```
npm run init
# or
bun run init
```

4. Accept any prompts and when it is done running, stop the application using Control + c
5. Run the application using this command

```
npm run serve
# or
bun run serve
```

Backend

Containers

This project requires Docker > v2 installed and running.

1. Clone the project from the github repo

2. From the project root, access the subfolders of the **infrastructure** and **services** folders, rename the .env.example files to .env and update the configuration in them to match.

   Replace the postgres database configuration in the services .env files with that for the live database.
   If one does not exist follow the steps in Code (Minimal) below before continuing

3. In the project root, open a terminal and run the containers using

   ```
   docker compose -f docker-compose.local.yml up --build -d
   ```

   Note: this takes a lot of time; approx 1 - 3 Hrs

4. Install the local root certificate located at **./infrastructure/certs/rootCA.pem** :

   - install mkcert
   - install the CA on your machine
   - install the CA on your browser

5. Ensure you can access the following documentation url endpoints on the browser (this might require waiting for a few minutes)

   - RAG Service: https://localhost:8003/docs
   - Ingest Service: https://localhost:8004/docs
   - Document Processor Service: https://localhost:8005/docs

6. To force a rebuild of the images used for the containers, run

   ```
   docker compose -f docker-compose.local.yml build --no-cache
   ```

7. To stop the running containers, run

   ```
   docker compose -f docker-compose.local.yml up --build -d
   ```

Code (Minimal)

This project requires Python >= 3.13 to run.

Recommendations:

- install UV and install Python using UV here
- setup a virtual environment and run the project in the virtual environment
- activate the virtual environment before following the steps below and before subsequent runs of the project

---

1. Clone the project from the github repo
2. From the project root, access the subfolders of the **infrastructure** and **services** folders, rename the .env.example files to .env and update the configuration in them to match.

   Replace the placeholder passwords and secrets as well as those in the database url and provide valid keys and ids for SharePoint and OpenAI in the .env files

3. In the project root directory, open a terminal and install the dependencies using

```
pip install -r ./requirements.txt
```

4. Run the setup scripts using:
   - for linux / mac os

```
sh ./infrastructure/scripts/setup_project_migrations.sh
```

   - for windows

```
bash ./infrastructure/scripts/setup_project_migrations.sh
```

Code (Full)

1. Follow the steps outlined in Code (Mininal) above.
2. To run a specific service:
   - Open a terminal at the root directory of the service eg. at ./services/rag-service/
   - Install the dependencies required for that service using

```
pip install -r ./requirements.txt
```

   - Update the ROOT_CERT_PATH in the .env file to the absolute path to ../../infrastructure/certs/rootCA.pem */certs/rootCA.pem is the accurate root cert path if the service is running in a container*
   - Run the service using

```
python -m uvicorn app.main:app
--host 127.0.0.1 --port 8000 --reload
```

```
--ssl-keyfile ../../infrastructure/certs/localhost+3-key.pem
--ssl-certfile ../../infrastructure/certs/localhost+3.pem
```

The port should be of the form 80xx

- The API Documentation for the service can now be accessed at https://host:port where from the above step, the host is 127.0.0.1 and the port is 8000

3. To run multiple services, repeat the step above but use a unique 80xx port for each, eg. 8000, 8001, 8002 ...
4. To allow interactions between the multiple services over http, update the following .env configurations in each service to match the host and port set for each service:
    - INGEST_SERVICE_ORIGIN

Project Run

1. Ensure Docker is running.
2. Open a terminal at the root directory of the backend project and run:

```
docker compose -f docker-compose.local.yml up --build -d
```

3. Ensure the local root certificate is installed in either or both your machine and default browser *see Project Setup -> Backend -> Containers*
4. Ensure you can access the following documentation url endpoints on the browser (this might require waiting for a few minutes)
    - RAG Service: https://localhost:8003/docs
    - Ingest Service: https://localhost:8004/docs
    - Document Processor Service: https://localhost:8005/docs
5. Open a terminal at the root directory of the frontend project and run:

```
npm run serve
# or
bun run serve
```

6. Ensure you're logged into a valid organization account for the frontend on your default browser
7. The frontend will launch on the browser when it is done loading